Programming for Economist

PS 8 – Random Numbers & Simulation

Class 3

Matias B.F. Hall,
Institute of Economics,
November 2026



KØBENHAVNS UNIVERSITET

Plan for today

1. Random numbers and simulations
2. Hints for PS8

1. Random numbers and simulations

Random numbers:

- A computer is never truly random. Instead the random number generators (rng) produce pseudo-random sequences.
- The stream is a sequence of numbers produced by the rng. The seed initialises the generator, and each draw advances its state.
  - Set the seed once and let the generator advance with each draw.
  - Only reset the seed when you want the exact same draw/same results.

Simulation:

- Simulation uses a model to generate possible outcomes. E.g. workers moving between employment and unemployment.
- Simulated outcomes vary across draws. Larger samples lead to less variation in aggregate estimates.
- Simulation often complements optimisation. First find optimal decisions, then simulate their consequences. Simulation can also feed back into optimisation: adjust behavioural parameters with each simulation, until the model outcomes better fit the data.
- Simulation helps us explore the implications of a model, but its results depend on the model's assumptions.

2. Hints for PS8

- Q2
  - Use plt.hist and plt.loglog to do the plotting
  - Use “1 - np.arange(1, size + 1) / size” for the stochastic y when plotting the tail.
- Q3
  - Start with simple logic – use loops!
    - Loop over time and individuals(Here N=1, but in Q4 N>1)
    - Use if statements to check what state the individual is in, in period t (employed or unemployed)
    - Depending on their draw, update their state in t+1. e.g. If draw < s, then state_{t+1} = U
    - The lecture solution is more efficient but a harder solution – start simple.
- Q4
  - Use your function from Q3
  - Use np.mean(‘states’, axis=1) to get mean for each period
  - Look at lecture solution, for a more efficient function.

2. Hints for PS3

- Q5
  - Use the difference equation iteratively using:
  - `for period in range(60):`
  - Save results to a list/numpy array within the loop with each iteration.
  - For the recession question, create a numpy array of separation rates using `np.full(T, s)`. Then set the rate to 0.04 between [5:5+8+1]. Use the array of rates instead of the integer, using the period to index.
- Q6
  - Optional:
    - No loops needed, only your knowledge about numpy broadcasting
    - Create a function that takes any number of etas to generate draws and returns
    - Use `NumpyArray[:, np.newaxis]` or `NumpyArray[:, None]` to broadcast two numpy arrays together with different dimensions.